Course Summary & What's Next

Week 13: CS 203 — Software Tools and Techniques for AI

Prof. Nipun Batra

IIT Gandhinagar · Spring 2026

The Journey

In January, you could write a Python script.

Now you can:

- **Collect** data from APIs, web pages, sensors
- **Validate** it with schemas before it poisons your model
- **Label** it efficiently (and know when to let an LLM help)
- **Augment** it when you don't have enough
- **Call LLMs** and stream structured outputs
- **Evaluate** models properly — not just accuracy
- **Tune** hyperparameters without wasting a week
- **Track** experiments so you never lose a good run
- **Dockerize** your work so anyone can reproduce it
- **Detect drift** when the world moves on without you
- **Profile & quantize** models for real-world constraints

One-Liner Per Lecture

#	Week	Takeaway in one sentence
1	Data Collection	*Never trust an API that doesn't return a status code; paginate, retry, cache.*
2	Data Validation	*Validate at the boundary — turn bad inputs into loud errors, not silent bugs.*
3	Data Labeling	*The annotation guidelines matter more than the tool; write them first.*
4	Optimizing Labeling	*Active learning + weak supervision turn labels from a headcount problem into a code problem.*
5	Data Augmentation	*Augmentation is free data — but only if the transform preserves the label.*
6	LLM APIs	*Prompts are functions; put them in version control and test their outputs like code.*
7	Model Evaluation	*A single train/test split is a coin flip; 5-fold CV is the minimum viable evaluation.*
8	Tuning + Tracking	*Random beats grid for ≥3 hyperparameters; Optuna beats random when evals are expensive.*
9	Reproducibility	*Control the machine (Docker), the math (seeds), and the memory (tracking) — all three.*
10	Data Drift	*The model you deployed is not the model you have tomorrow; monitor distributions, not just metrics.*
11	Profiling + Quantization	*Profile before you optimize; INT8 is ~4× free compression on every Linear layer.*

The Full Stack, One Slide

```
Raw Data
  │
  ├── Collect (APIs, scraping, sensors)     ← Week 1-2
  ├── Validate (schemas, types, ranges)     ← Week 2
  ├── Label (manual, active, weak, LLMs)    ← Week 3-4
  ├── Augment (transforms, synthetic)       ← Week 5
  │
  ├── Call LLM APIs (Gemini, prompts)       ← Week 6
  ├── Train & Evaluate (CV, bias-variance)  ← Week 7
  ├── Tune & Track (Optuna, TrackIO)        ← Week 8
  │
  ├── Reproduce (Docker, seeds)             ← Week 9
  ├── Monitor (drift detection)             ← Week 10
  ├── Profile & Optimize (INT8, ONNX)       ← Week 11
  ├── Build Agents (tools, loops)           ← Week 12
  │
  └── Production ML System
```

Each layer above depends on the ones below. Skip one, ship a bug.

Part I — Data (Weeks 1–5)

"80% of machine learning is data work."

Data Collection: What We Learned

APIs — structured data from services (JSON, REST)

```
r = requests.get("https://api.example.com/data", timeout=5)
r.raise_for_status()   # crash loudly, don't silently corrupt
data = r.json()
```

Web Scraping — extracting data from HTML pages

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(html, "html.parser")
titles = [h.get_text().strip() for h in soup.find_all("h2")]
```

Key insight: The data you collect determines your model's **ceiling**.
No algorithm fixes bad data.

Gotchas you met this semester: rate limits, pagination, JavaScript-rendered pages, character encoding, time-zone bugs.

Data Validation: What We Learned

Pydantic — validate with Python type hints

```
class SensorReading(BaseModel):
    temperature: float = Field(ge=-50, le=60)
    humidity:    float = Field(ge=0, le=100)
    timestamp:   datetime
```

Why this matters: a bad row at 3 AM will otherwise poison your training set and you will only notice two weeks later when accuracy drops.

Key insight: Validate at the **boundary**. Catch bad data before it enters your pipeline — *not* after your model produces garbage.

Labeling: The Bottleneck

Strategy	When to use	Cost / sample
Manual	Small, high-stakes data	Highest
Active learning	Large unlabeled pool, uncertain model	Medium
Weak supervision	Domain heuristics, noisy labels	Low
LLM-assisted	Text tasks, budget for API calls	Low but ≠ free

Active learning in one line:

$$\text{next_to_label} = \arg\max_{x \in \mathcal{U}} \; H\big(p(y \mid x)\big)$$

— pick the most *uncertain* unlabeled example. You annotate 10× fewer samples for the same accuracy.

Weak Supervision in 10 Lines

```python
def lf_has_positive_word(text):
    return 1 if any(w in text for w in ["great", "excellent", "love"]) else -1

def lf_has_negative_word(text):
    return 0 if any(w in text for w in ["bad", "terrible", "hate"]) else -1

# Majority-vote aggregation
labels = [lf_has_positive_word(t), lf_has_negative_word(t), ...]
noisy_label = Counter(l for l in labels if l ≠ -1).most_common(1)[0][0]
```

Key insight: You can get 70–90% of manual-labeling accuracy with hand-written rules + a denoiser. That's the Snorkel playbook.

Augmentation: Free Data (*if* label-preserving)

Domain	Techniques
Images	flip, rotate, crop, color jitter, RandAugment
Text	synonym replacement, back-translation, paraphrase
Audio	time shift, pitch shift, noise injection
Tabular	SMOTE, noise injection, mixup

⚠️

The canonical mistake: flipping a handwritten `6` and calling it a `9`. Always ask: *does this transform preserve the label?*

Part II — Models (Weeks 6–8)

"A simple model you can trust beats a complex one you can't."

LLM APIs: What We Learned

```python
from google import genai
client = genai.Client()

response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents=prompt,
    config={"response_mime_type": "application/json",
            "response_schema": ReviewSentiment},
)
```

- **Structured outputs** via JSON schema
- **Multimodal** — images + text in one call
- **Streaming** for UX
- **Prompts are code** — version them, test them

Key insight: Prompt engineering ≠ magic. It's testable software.

When *Not* to Use an LLM

Task	Better choice
Exact arithmetic	A calculator
Date parsing	`datetime.strptime`
Structured extraction from text	regex or a small fine-tuned model
Classification with ≥1000 labelled examples	logistic regression / BERT
Anything that must be deterministic	Pretty much anything else

LLMs are a tool, not a default.

Evaluation: Beyond Accuracy

Concept	Why it matters
Cross-validation	Single split = coin flip
Stratified K-Fold	Preserves class balance
Bias–Variance	Underfitting vs overfitting, visualized
Leakage	#1 cause of too-good-to-be-true results
Nested CV	Needed when you tune hyperparameters

Key insight: If you only do one thing, do **5-fold stratified cross-validation** instead of one train/test split. That's the floor.

Bias-Variance in One Formula

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{bias}^2} + \underbrace{\mathrm{Var}[\hat{f}(x)]}_{\text{variance}} + \sigma^2$$

- Simple models → high bias, low variance (underfit)
- Complex models → low bias, high variance (overfit)
- The "sweet spot" is where **test error** bottoms out, not training error

Regularization, more data, and CV all attack the variance term.

Data Leakage: The #1 Killer

Any time **test-time information** sneaks into training:

- Scaling on the full dataset *before* the train/test split
- Using a target-derived feature (`log_of_price` to predict `price`)
- Time-series data split randomly instead of by time
- Duplicated rows across train and test

Rule of thumb: if your test accuracy is 99% on a problem everyone else solves at 85%, you have a leak. Look harder.

Tuning: Don't Waste Time

Method	Tries	When
Grid Search	all combos	< 3 hyperparams
Random Search	random subset	3+ hyperparams
Bayesian (Optuna)	adaptive	expensive evals
Hyperband / ASHA	early stopping	deep learning

Bergstra & Bengio 2012 showed **random beats grid** because only ~2 of your 5 hyperparameters actually matter, and random explores those coordinates more densely.

Experiment Tracking in 4 Lines

```python
import trackio
run = trackio.init(project="moons", config={"lr": 1e-3, "hidden": 16})
for epoch in range(100):
    trackio.log({"loss": loss.item(), "acc": acc.item()})
run.finish()
```

- Never lose a good run again
- Sweeps + comparison dashboards
- Hyperparameter importance via TPE / fANOVA

Key insight: *"Which run was the good one?"* is the question tracking answers. Without it, you're re-running experiments by hand.

Part III — Production (Weeks 9–11)

"A model in a notebook is not a model."

Reproducibility: Three Layers

Layer	Symptom	Cure
The Math	Different results each run	seeds (`numpy` , `torch` , `random`)
The Memory	Lost the best hyperparams	experiment tracking (TrackIO)
The Machine	"Works on my laptop"	Docker / `pyproject.toml`

```
# The Math
import random, numpy as np, torch
random.seed(0); np.random.seed(0); torch.manual_seed(0)

# The Memory
trackio.init(project="proj"); trackio.log({"acc": 0.95})

# The Machine
# Dockerfile → docker build -t app . → docker run -p 7860:7860 app
```

Docker — The 30-Second Explanation

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

- `docker build` → one command turns your code into a portable image
- `docker run -p 7860:7860` → ships to any Linux box, Mac, CI server
- No more *"can you install these 14 things first"*

Key insight: Docker isn't about containers, it's about **recipes that always work.**

Data Drift: Is Your Model Still Good?

Models degrade silently as the world changes.

Test	Detects
KS test	Shift in continuous features
PSI	Population stability between train and prod
Chi-squared	Categorical feature shifts
Jensen–Shannon / Wasserstein	General distribution distance

$$\mathrm{KS} = \max_x |F_{\text{train}}(x) - F_{\text{prod}}(x)|$$

Key insight: 95% accuracy at deployment can silently drop to 70% two months later. Monitor **distributions**, not just metrics.

Profiling: Find Before You Fix

```
print(time.time() - t0)   # first check: how slow?
cProfile                  # which function?
line_profiler             # which line?
memory_profiler           # which allocation?
torch.profiler            # GPU too
```

Key insight: The #1 speedup in most student projects this semester was *loading the model once at startup* instead of once per request.

Premature optimization is the root of all evil — and the #1 cause of bugs.

Quantization in Three Lines

$$\text{scale} = \frac{\max|W|}{127}, \qquad q = \text{round}(W/\text{scale}), \qquad \hat{W} = q \cdot \text{scale}$$

```python
def quantize_tensor(x):
    s = x.abs().max().item() / 127.0
    q = torch.round(x / s).clamp(-127, 127).to(torch.int8)
    return q, s
```

4× smaller weights, <1% typical accuracy drop, runs anywhere.

Quantization Scales from MLP → 0.5B LLM

We applied the same three-line quantizer to three models, in sequence:

Notebook	Model	Params	INT8 accuracy drop
`09-quantization-from-scratch`	2-layer MLP, `make_moons`	~400	≈ 0
`11-quantization-llm-from-scratch`	2-layer Transformer, Hamlet corpus	~60K	~0.005 loss
`12-quantization-real-llm`	**Qwen 2.5 0.5B** (Hugging Face)	**494M**	**< 1% CE**

The same 3 lines work on a toy and a production LLM. That's the whole point.

Part IV — Agents (Week 12)

"The same pattern that builds Claude Code builds yours."

Agents: LLMs That Take Actions

$$\mathbf{agent} = \mathbf{LLM} + \mathbf{tools} + \mathbf{loop}$$

Piece	Role	Analogy
LLM	Thinks, reasons, decides	The brain
Tools	Functions the LLM can call	The hands
Loop	Keep going until done	The work ethic

You built a full agent with **Gemma 4** on a free Colab T4: four tools, multi-step reasoning, **~100 lines** of Python.

Tool Calling — The Minimum Viable Agent

```python
def calculate(expr: str) -> str:
    return str(eval(expr, {"__builtins__": {}}, SAFE_FUNCS))

TOOLS = [{
    "type": "function",
    "function": {"name": "calculate", "description": "Do math",
                 "parameters": {"type": "object",
                                "properties": {"expr": {"type": "string"}}}}
}]

while not done:
    response = llm(messages, tools=TOOLS)
    for call in response.tool_calls:
        result = dispatch(call)
        messages.append({"role": "tool", "content": result})
```

Key insight: Claude Code, Cursor, Devin, Perplexity — all the same pattern. Bigger tools, same architecture.

Connecting the Dots

The Tools Map

```
PLAN            BUILD             SHIP              EXTEND
────            ─────             ────              ──────
Validation      sklearn           Docker            cProfile
Schemas         Pipelines         TrackIO           Quantization
Labeling        CV / Tuning       Seeds             ONNX
Augmentation    Optuna            Drift detection   Pruning
                LLM APIs          CI/CD             Distillation
                HF Transformers   Gradio/Streamlit  Agents
                                  FastAPI           Tool calling
```

You don't need all of these for every project — but you need to know they **exist** so you can pick the right tool for the job.

Common Mistakes I Saw This Semester

1. **Scaling before splitting** → data leakage → fake accuracy
2. **No seed set** → "my model worked yesterday"
3. **One train/test split** reported as final accuracy → coin flip
4. **Pickled the model without requirements.txt** → unreproducible
5. **Hardcoded paths** (`/home/student/...`) → breaks for the next person
6. **No README** → nobody can run it
7. **Test set peeking** during hyperparameter tuning → optimistic bias
8. **Committed API keys / secrets** → please read `.gitignore` docs
9. **Caught every exception broadly** → silent failures
10. **Optimized the wrong bottleneck** — profiling would have shown the fix in 30 seconds

If you avoid just these 10, you're already ahead of most production codebases.

The Minimum Viable ML Project

```
my-project/
├── data/                  # raw + processed data (gitignored)
├── notebooks/             # exploration only
├── src/
│   ├── train.py          # training, with seeds + tracking
│   ├── evaluate.py       # reproducible metrics
│   └── app.py            # Gradio/Streamlit/FastAPI demo
├── tests/                 # at least one smoke test
├── requirements.txt       # pinned versions
├── Dockerfile             # optional but impressive
└── README.md              # how to run it in < 5 min
```

Must have: seeds set, **5-fold CV**, TrackIO logging, working demo.

What This Course Didn't Cover

Topic	Why it matters	Where to learn
Deep Learning	CNNs, transformers, fine-tuning	CS 337, fast.ai
MLOps	CI/CD, model registries, pipelines	*Made With ML*
Cloud Deployment	AWS/GCP/Azure, k8s	cloud docs
Data Engineering	ETL, warehouses	`dbt`, Airflow
ML System Design	Scaling, A/B tests, feedback loops	Chip Huyen's book
MCP / A2A	Universal tool standards for agents	Anthropic / Google docs

This course gave you the **foundations**. These are the next steps.

Cheat Sheet — *"I have a problem, which slide?"*

Symptom	Diagnosis	Lecture
"Accuracy changes every run"	No seed set	9
"Accuracy dropped in production"	Data drift	10
"Test accuracy 99%, prod 70%"	Data leakage	7
"Model is too slow"	Profile first	11
"Model too big for device"	Quantize (INT8)	11
"LLM hallucinates numbers"	Give it a calculator tool	12
"I lost my best hyperparams"	TrackIO	8
"My script doesn't run for the TA"	Docker + requirements.txt	9
"I can't get enough labels"	Active learning / weak supervision	4
"Not enough data to train"	Augmentation	5

Screenshot this slide. Come back to it in 5 years.

Advice for Your Future Projects

1. **Start with the data, not the model.** 80% of ML is data work.
2. **Simple models first.** Logistic regression before transformers. You'll be surprised how often simple wins.
3. **Automate the boring stuff.** Seeds, tracking, Docker, `requirements.txt` — set these up on day 1, not day last.
4. **Make it runnable.** If someone can't clone your repo and run your code in 5 minutes, it doesn't exist.
5. **Read error messages.** The answer is almost always in the traceback. Read it before asking ChatGPT.
6. **Version everything.** Code, data, models, configs, prompts.
7. **Ship early, iterate.** A 70% demo today beats a 95% plan for next month.

What to Do Monday Morning at a New ML Job

1. Read the existing codebase. Find the `train.py`. Run it. Does it reproduce? *That alone impresses people.*
2. Ask: "What's the source of truth for the training data?" If the answer is "this one CSV in somebody's Downloads folder", you have work to do.
3. Look for seeds. Add them if missing.
4. Check if experiments are tracked. Add TrackIO / W&B if not.
5. Write a Dockerfile. It won't be perfect. Ship it anyway.

You now know enough to be **the person** who does these things.

Thank You

This was **CS 203: Software Tools and Techniques for AI**.

You started with scripts.
You end with reproducible, monitored, optimized ML systems — and agents that can take actions.

```
Collect → Validate → Label → Augment
          ↓
  Train → Evaluate → Tune → Track
          ↓
Reproduce → Monitor → Profile → Quantize
          ↓
        Agent → Production
```

Go build something.

Slides, code, videos: https://nipunbatra.github.io/stt-ai-teaching/

Questions?

From scripts to systems. Go build.